
КАФЕДРА

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

должность, уч. степень, звание

подпись, дата

инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ
Машинное обучение: оценка качества минимально жизнеспособного
продукта
по курсу: Методы исследований в менеджменте

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

подпись, дата

инициалы, фамилия

Санкт-Петербург 2025

Цель работы:

Освоить применение методов машинного обучения к задачам многоклассовой классификации в MATLAB (приложение Classification Learner): подготовка синтетической обучающей выборки, настройка кросс-валидации, обучение и сравнение нескольких семейств моделей (деревья, LDA, k-NN, SVM и др.), оценка качества (Confusion Matrix, ROC/TPR-FPR, предсказания), экспорт лучшей модели и прогноз на новых данных. Реализовать аналогичную процедуру для задачи конкурентного окружения (5 признаков, 3 класса).

Задание:

1. Сформировать обучающую выборку из 45 наблюдений (по 15 на класс):
 - для MVP: $X_1 \dots X_5 \rightarrow \text{класс } 1/2/3$;
 - для конкурентного окружения: Audience, NewEntrants, Suppliers, Substitutes, Rivalry $\rightarrow$ «благоприятная/средняя/неблагоприятная».
2. Загрузить таблицу в Classification Learner, указать Y как Response, предикторы — 5 признаков, включить 5-fold cross-validation.
3. Обучить набор моделей (деревья, LDA, k-NN, SVM, логистическая регрессия, нейросети и др.), сравнить по Accuracy (Validation).
4. Проанализировать Confusion Matrix, ROC-кривые, Scatter (Model predictions) и Parallel Coordinates.
5. Выбрать модель-победитель, Export Compact Model, выполнить прогноз predictFcn(Xnew) для нового наблюдения.
6. Повторить пункты 2–5 для задачи конкурентного окружения.

Выполнение задания:

Теоретическая часть и знакомство с инструментом

Теоретические основы и знакомство с приложением Classification Learner

Формирование обучающей выборки

Это синтетическая (разыгранная) выборка из 45 наблюдений про качество минимально жизнеспособного продукта (MVP).

Признаки (X1–X5):

- X1 – скачивания, тыс. установок (чем больше, тем лучше).
- X2 – пользовательский рейтинг в магазине, 1–10 (чем выше, тем лучше).
- X3 – размер приложения, МБ (10-40; меньше – лучше, т.к. большой размер отпугивает часть пользователей).
- X4 – доля платящих клиентов, % (1-10; больше – лучше).
- X5 – отток пользователей, % (5-30; меньше – лучше).
- Отклик Y – категория качества MVP: 1 = высокая, 2 = средняя, 3 = низкая.

Интуитивная логика такая: много скачиваний и высокий рейтинг + небольшой размер + высокий % платящих + низкий отток → класс 1 (высокая); обратная комбинация → класс 3 (низкая); промежуточные значения → класс 2 (средняя).

Генерируем обучающую выборку (45×6):

Сгенерирована синтетическая выборка (метод Монте-Карло), 45 наблюдений: 5 признаков X1–X5 и целевая переменная Y (1 – высокая, 2 – средняя, 3 – низкая оценка MVP). Диапазоны: X1 – тыс. скачиваний, X2 – рейтинг 1-10, X3 – размер 10-40 МБ, X4 – доля платящих 1-10 %, X5 - отток 5-30 %. Классы сбалансированы: по 15 точек.

	A	B	C	D	E	F
	mvpdata					
	X1	X2	X3	X4	X5	Y
	Number	Number	Number	Number	Number	Number
1	X1	X2	X3	X4	X5	Y
2	53.0597...	7.65280...	20.2216...	8.50940...	7.34841...	1
3	83.3474...	8.43881...	29.6638...	6.91553...	9.30469...	1
4	87.0921...	9.38217...	22.3052...	7.09357...	7.92142...	1
5	64.6903...	7.67290...	26.9402...	6.27336...	11.7249...	1
6	62.7440...	7.52586...	16.5762...	7.55733...	9.29059...	1
7	58.1010...	8.57746...	19.7505...	7.87632...	11.8309...	1
8	72.1065...	9.86680...	21.8256...	8.75163...	11.5380...	1
9	66.2822...	8.09663...	25.2054...	6.26698...	6.86609...	1
10	90.9250...	7.07854...	27.8327...	7.01568...	8.22880...	1
11	78.2390...	9.39668...	20.9969...	6.11706...	9.31919...	1
12	65.0161...	9.55881...	27.9735...	8.99415...	9.22367...	1
13	78.2427...	7.61049...	22.1584...	9.17860...	7.95391...	1
14	48.0126...	8.21920...	25.7618...	6.64066...	9.30136...	1
15	76.7320...	7.20938...	27.3081...	9.12936...	9.56160...	1
16	45.2577...	6.28587...	21.9690...	7.36104...	9.91686...	1
17	41.5463...	4.91391...	34.1218...	3.64023...	19.1555...	2
18	34.4939...	4.27516...	33.6292...	5.33384...	13.3094...	2
19	56.7563...	6.73024...	22.5934...	5.31688...	16.6887...	2
20	31.3542...	6.55816...	22.6916...	3.83345...	14.9914...	2
21	39.2107...	5.88515...	25.4078...	5.37281...	18.6161...	2
22	57.7636...	4.82803...	27.1685...	4.54871...	16.2331...	2
23	32.5545...	5.04015...	31.4532...	4.60227...	11.3767...	2
24	37.8498...	6.35714...	27.9062...	4.58864...	13.2857...	2
25	45.6747...	5.37635...	23.9033...	3.81672...	12.6500...	2
26	39.1044...	6.67540...	28.5006...	6.80751...	16.0001...	2
27	42.5812...	5.66325...	25.9215...	4.62768...	11.2193...	2
28	36.2431...	5.40163...	25.9056...	5.01702...	13.8888...	2
29	35.9123...	7.35599...	23.5363...	3.50305...	14.9402...	2
30	34.5220...	5.00471...	33.6538...	5.01376...	17.0965...	2
31	36.4658...	4.71403...	22.5585...	4.61595...	10.7776...	2
32	27.0758...	3.58508...	36.1865...	1.85611...	18.7597...	3
33	26.7194...	2.94357...	35.2167...	2.65203...	20.1522...	3
34	13.4388...	4.10651...	36.6987...	3.59877...	20.2207...	3
35	18.1885...	1.33131...	31.9090...	2.73372...	21.9760...	3
36	31.2411...	3.59116...	35.7901...	2.35454...	23.7958...	3
37	20.4085...	2.70965...	33.6731...	3.04306...	25.8089...	3
38	19.5215...	2.05128...	38.3808...	3.80425...	21.3923...	3
39	32.6042...	2.92217...	39.8376...	2.51355...	27.2512...	3
40	33.0501...	2.75010...	39.1657...	1.93362...	17.9793...	3
41	23.6695...	3.41447...	33.0839...	4.06429...	24.4368...	3
42	26.9057...	5.47542...	33.5330...	3.01287...	21.1780...	3
43	21.1321...	2.23503...	37.1900...	4.37038...	23.5585...	3
44	22.3187...	3.61567...	33.9108...	2.27349...	21.9415...	3
45	30.5394...	3.55701...	32.3598...	1.77029...	19.1725...	3
46	27.2511...	4.80487...	35.8632...	3.08302...	27.4240...	3

Рисунок 1 – Сгенерированные данные mvpData (45×6) в MATLAB (Variable Editor)

Таблица с признаками X1-X5 и меткой Y.

Подтверждает корректное создание набора: объём (45×6), типы столбцов, диапазоны значений, наличие трёх категорий класса.

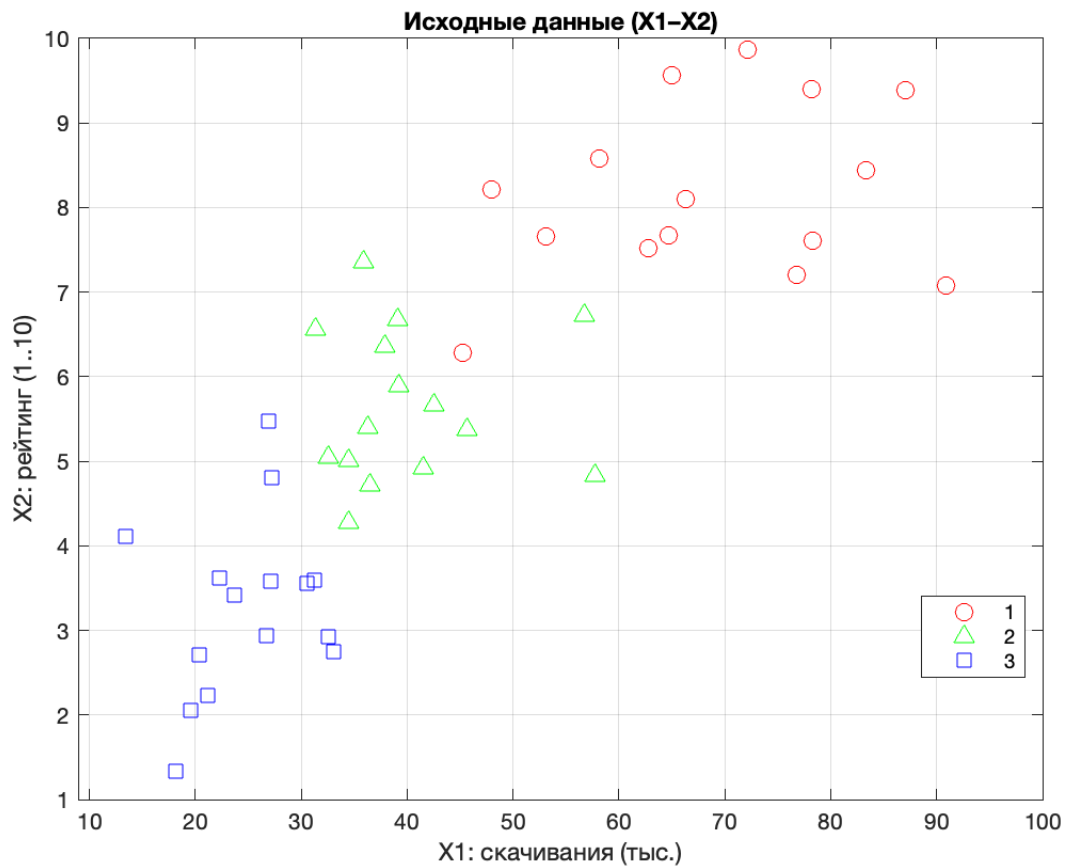


Рисунок 2 – Исходная выборка в проекции X1-X2 (цвет – класс Y: 1=высокая, 2=средняя, 3=низкая)

Рассеяние точек по двум признакам (скачивания и рейтинг); классы частично перекрываются.

Первичная визуальная диагностика. Видно, общую тенденцию «больше X1 и выше X2 → лучше класс», но из-за перекрытия понадобятся модели, способные разделять сложные границы (SVM), а не только простые пороги.

Создание сессии Classification Learner и первичный обзор данных

В приложении Classification Learner выбран набор mvpData (45×6), целевая переменная Y (категориальная, 3 класса), предикторы X1-X5. Для контроля переобучения задана 5-кратная перекрёстная проверка (5-fold Cross-Validation); отдельный тест-набор не выделялся.

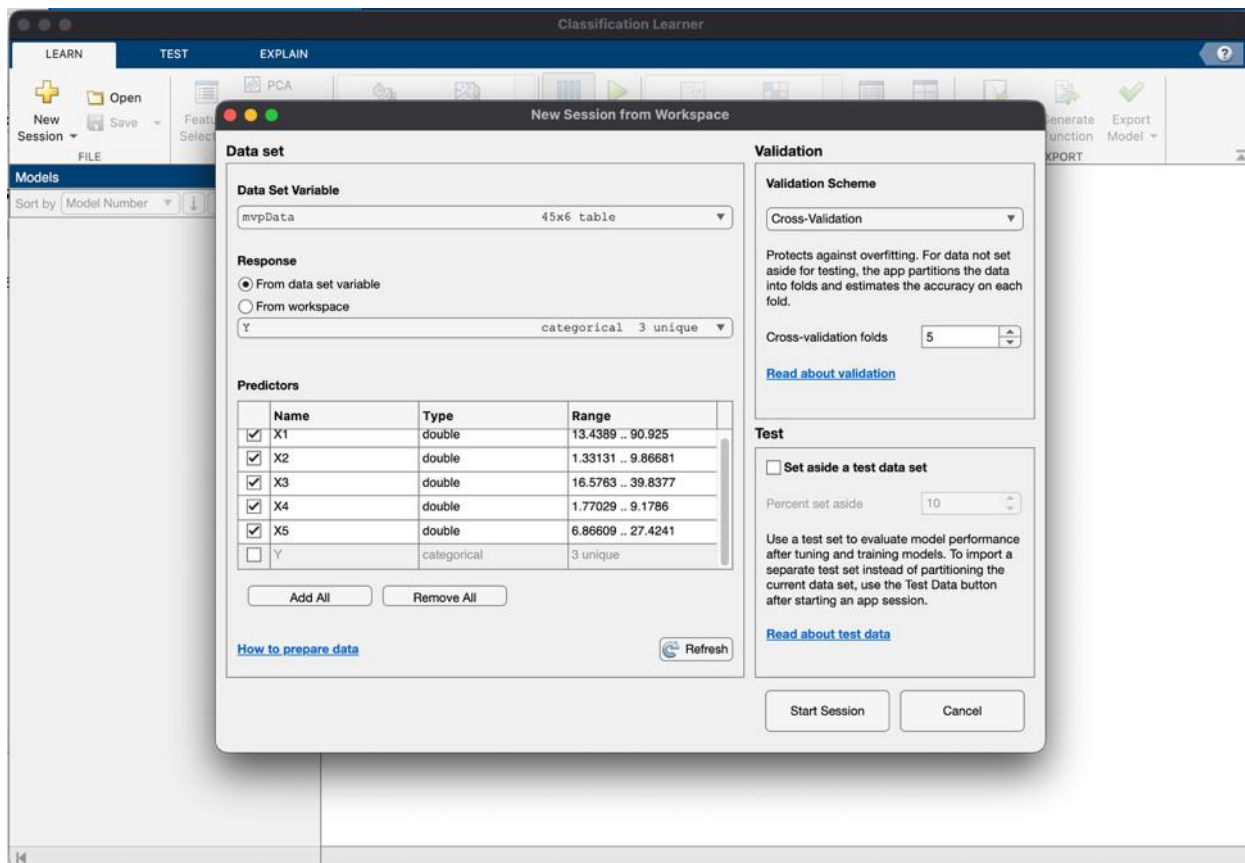


Рисунок 3 – Окно “New Session from Workspace”

Выбран Data Set Variable: mvpData (45×6), Response: Y (categorical, 3 unique), в списке Predictors: X1-X5 (тип double, указаны рабочие диапазоны). В блоке Validation задана схема Cross-Validation, folds = 5; опция Set aside a test data set отключена.

Фиксирует корректный импорт данных и настройки проверки модели, которые будут одинаково применены ко всем обучаемым классификаторам.

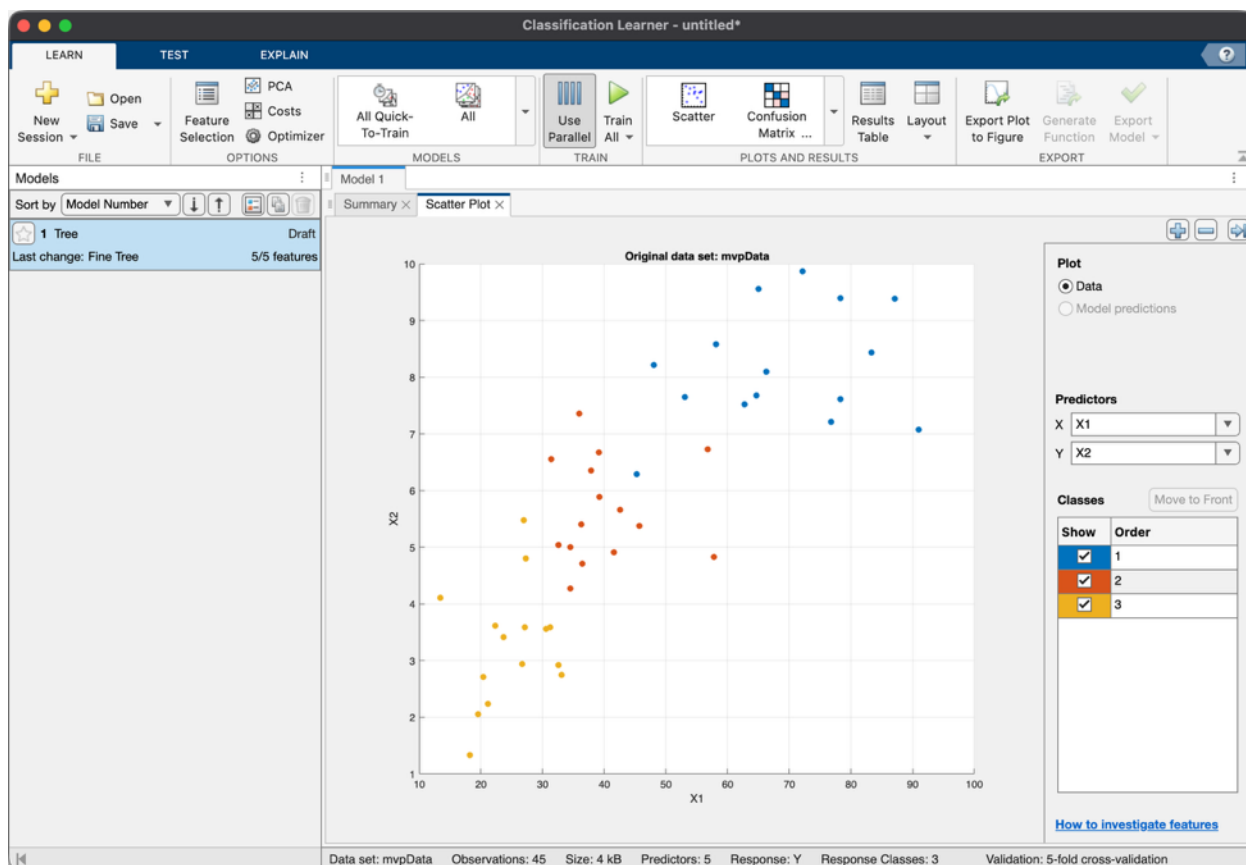


Рисунок 4 – Classification Learner: диаграмма рассеяния исходной выборки (X1–X2)

Вкладка Scatter Plot; по осям выбраны X1 (тыс. скачиваний) и X2 (рейтинг 1-10). Цвет точек – класс Y (1 – высокая, 2 – средняя, 3 – низкая оценка MVP). В правой панели отмечены отображаемые классы.

Быстрая диагностика разделимости: наблюдается тенденция «выше X1 и X2 → лучше класс», но есть перекрытие классов, значит понадобятся методы с нелинейными/гибкими границами (например, SVM), а не простые пороги.

Обучение набора моделей (ручной режим) и выбор лучшей

В приложении Classification Learner в ручном режиме были обучены разные классификаторы (деревья решений, LDA, k-NN, SVM и др.) на выборке mvprData. Схема проверки - 5-кратная перекрёстная валидация без выделения отложенного теста. По метрике Validation Accuracy наилучший результат показали Kernel Logistic Regression и SVM (Kernel) – обе модели дали 100% точности на кросс-валидации; в качестве финальной для анализа выбрана SVM (Kernel).

The screenshot displays the Classification Learner interface. The 'LEARN' tab is active, showing a list of models sorted by 'Accuracy (Validation)'. The top models are Kernel Logistic Regression (2.33) and SVM (Kernel) (2.32), both achieving 100.0% validation accuracy. The right panel shows the 'Summary' for Model 2.32: Kernel, which is trained and achieved 100.0% accuracy with 0 error rate. The bottom status bar indicates the data set is mvprData with 45 observations and 5 predictors.

Model	Accuracy (Validation)	Last change	Features
2.33 Kernel	100.0%	Logistic Regression Kernel	5/5 features
2.32 Kernel	100.0%	SVM Kernel	5/5 features
2.25 Ensemble	100.0%	Subspace KNN	5/5 features
2.14 SVM	100.0%	Medium Gaussian SVM	5/5 features
2.12 SVM	100.0%	Cubic SVM	5/5 features
2.11 SVM	100.0%	Quadratic SVM	5/5 features
2.7 Efficient Linear SVM	100.0%	Efficient Linear SVM	5/5 features
2.6 Efficient Logistic Regression	100.0%	Efficient Logistic Regression	5/5 features
2.31 Neural Network	97.8%	Trilayered Neural Network	5/5 features
2.30 Neural Network	97.8%	Bilayered Neural Network	5/5 features
2.24 Ensemble	97.8%	Subspace Discriminant	5/5 features
2.21 KNN	97.8%	Weighted KNN	5/5 features
2.20 KNN	97.8%	Cubic KNN	5/5 features
2.17 KNN	97.8%	Medium KNN	5/5 features
2.16 KNN	97.8%	Efficient KNN	5/5 features

Model 2.32: Kernel
Status: Trained

Training Results

Accuracy (Validation)	100.0%
Total cost (Validation)	0
Error rate (Validation)	0.0%
Prediction speed	~530 obs/sec
Training time	4.5198 sec
Model size (Compact)	~36 kB

Model Hyperparameters

- Feature Selection: 5/5 individual features selected
- PCA: Disabled
- Misclassification Costs: Default
- Optimizer: Not applicable

Data set: mvprData Observations: 45 Size: 4 kB Predictors: 5 Response: Y Response Classes:

Рисунок 5 – Список обученных моделей и сводка по лидеру

Слева – перечень обученных моделей, отсортированный по Accuracy (Validation); справа – вкладка Summary для выбранной модели SVM (Kernel): Accuracy (Validation) = 100%, 5/5 признаков, Cross-Validation: 5 folds, время обучения и размер компактной модели.

Фиксирует, какая именно модель признана лучшей и по какому критерию (валидационная точность при одинаковой схеме CV).

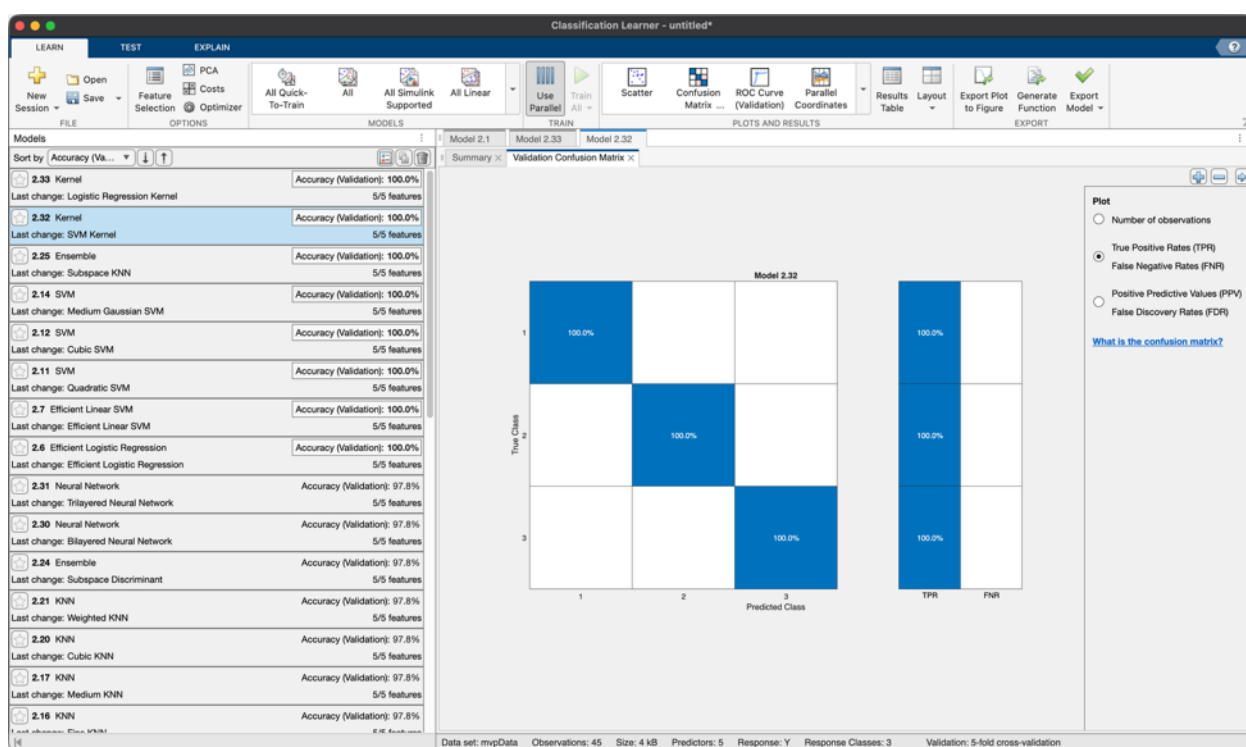


Рисунок 6 – Матрица ошибок (Confusion Matrix) для лучшей модели (5-fold CV)

Все наблюдения лежат на диагонали матрицы, вне диагонали – нули, класс-по-классу ошибок нет.

Подтверждает отсутствие заметных ошибок распознавания на кросс-валидации и показывает, что ни один из классов $Y=1/2/3$ не «проседает».

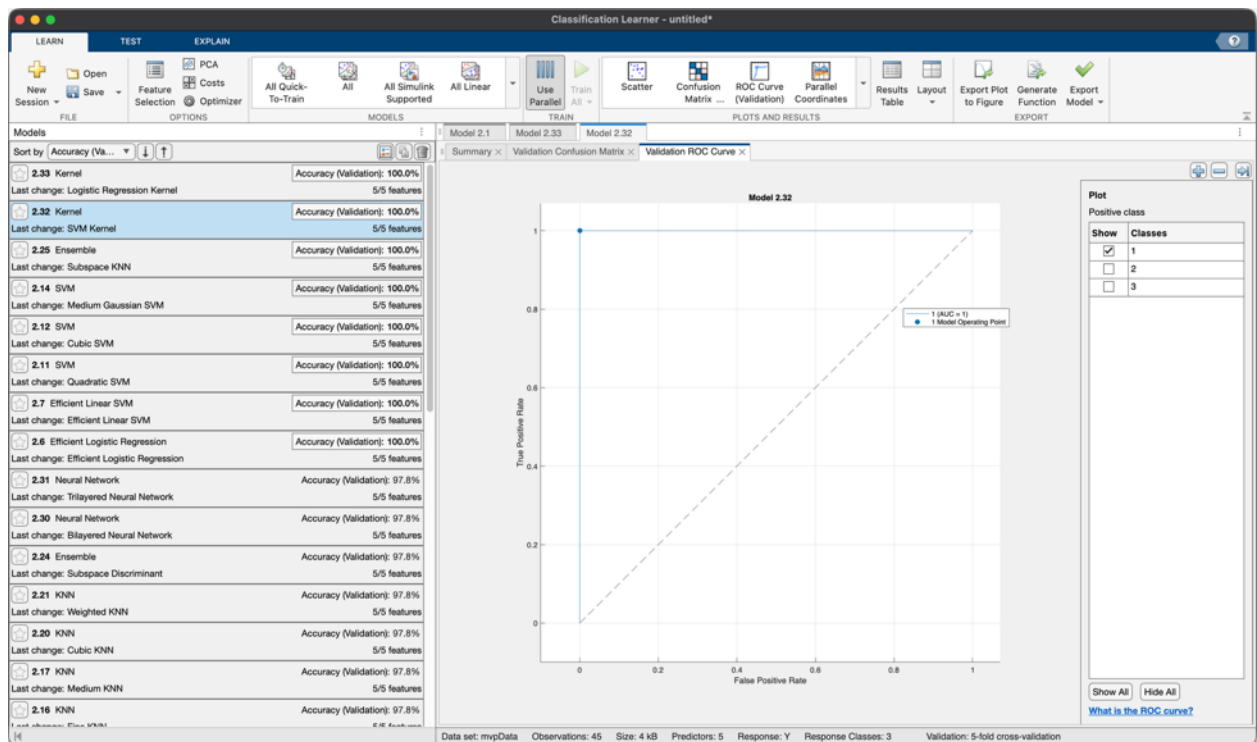


Рисунок 7 – ROC-кривые (one-vs-all) для классов $Y=1,2,3$

Каждую кривую, видно, выше диагонали случайной классификации; в текущем запуске маркеры лежат практически в точке $TPR=1$, $FPR \approx 0$, площадь под кривой $AUC \approx 1.0$.

Демонстрирует качество вероятностных предсказаний модели по каждому классу; высокая AUC говорит, что модель хорошо разделяет классы при разных порогах.

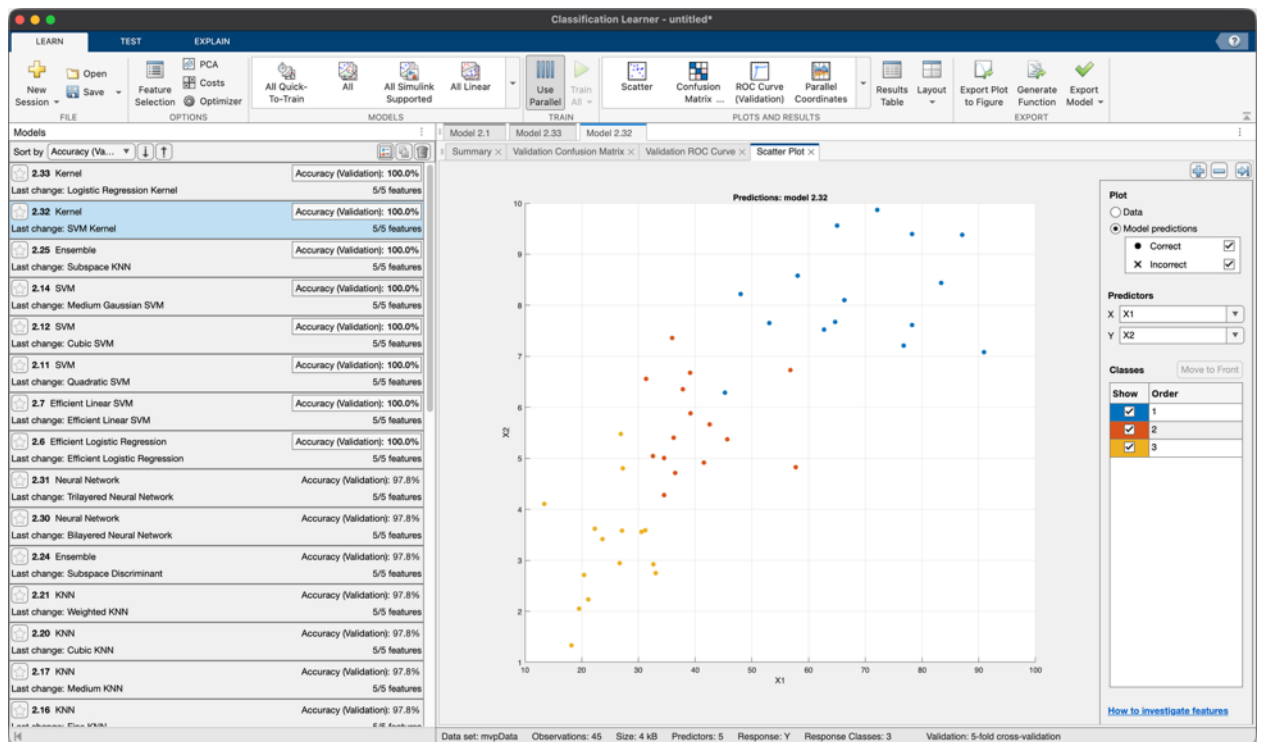


Рисунок 8 – Прогнозы выбранной модели (SVM, проекция X1-X2)

Диаграмма рассеяния исходных наблюдений, окраска – предсказанный класс (Model predictions). Включены флажки Correct/Incorrect – при наличии ошибок точки-ошибки помечаются крестиками; на данном рисунке крестиков нет.

Визуальная проверка, что модель корректно разделяет классы в проекции X1-X2 и что заявленная точность CV подтверждается отсутствием ошибок на этой проекции.

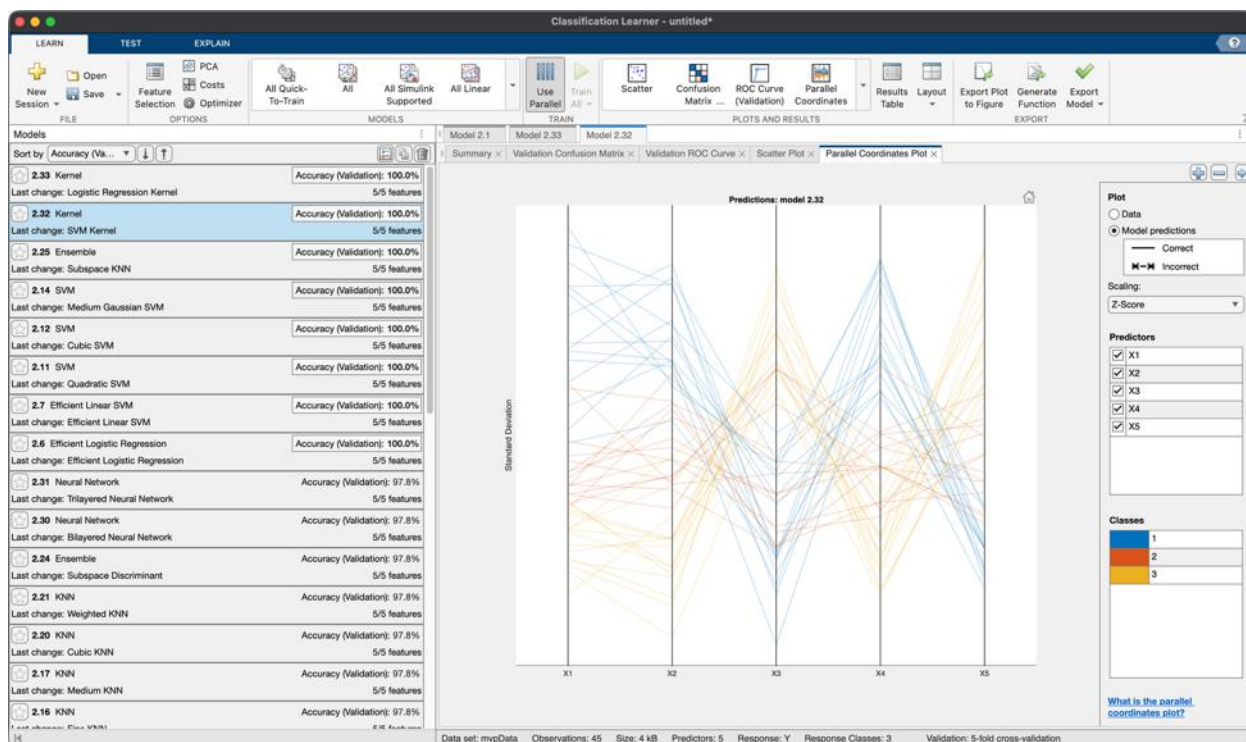


Рисунок 9 – График параллельных координат (z-score по X1-X5)

Стандартированные значения всех пяти признаков; каждая ломаная – одно наблюдение, цвет – класс

- Класс 1 (высокий MVP) — высокие X1 (скачивания), X2 (рейтинг), X4 (% клиентов), низкие X3 (объём) и X5 (отток).
- Класс 3 (низкий MVP) — противоположный профиль: низкие X1, X2, X4 и высокие X3, X5.
- Класс 2 (средний MVP) — промежуточные значения по большинству осей.

Такой разлёт по X1, X2, X4 и X5 объясняет, почему ядровые модели получили идеальную разделимость.

Пример другой модели (хуже):

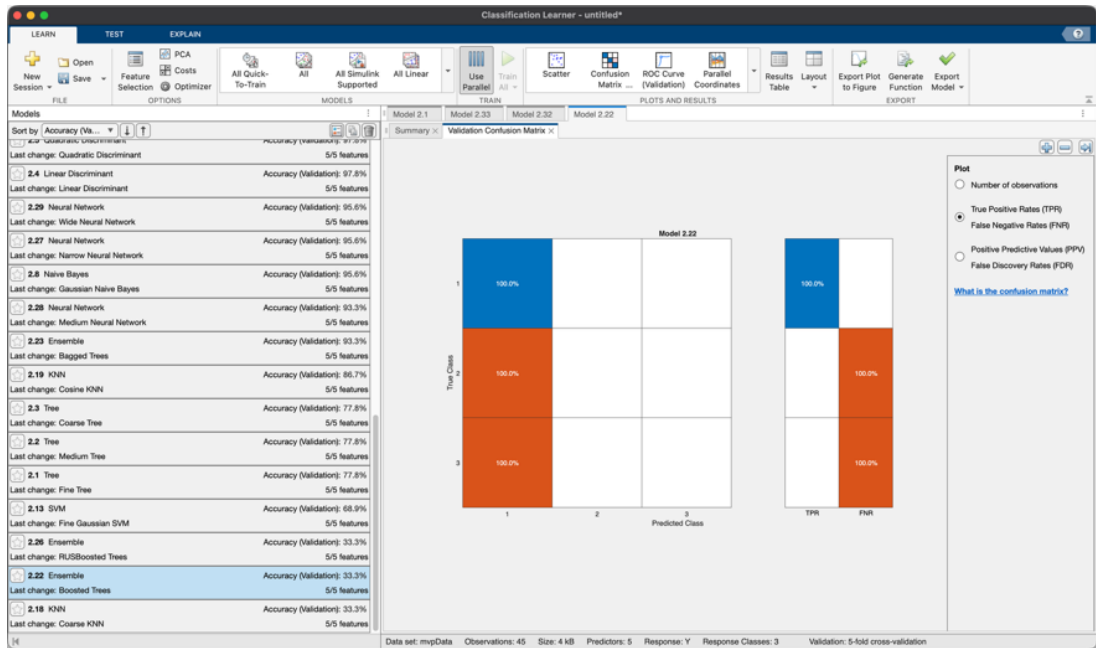


Рисунок 10 – Матрица ошибок

Ошибок заметно больше по вне-диагональным ячейкам, особенно для классов 2 и 3.

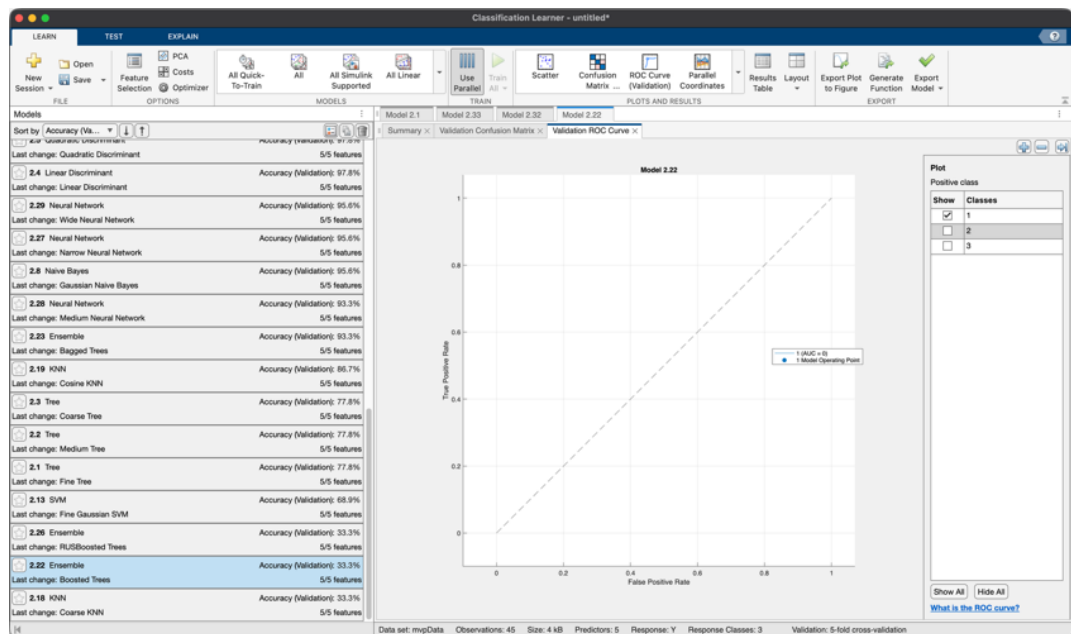


Рисунок 11 – ROC-кривая

Кривая ближе к диагонали случайного угадывания; площадь под кривой меньше, чем у лучшей модели → слабее разделение классов.

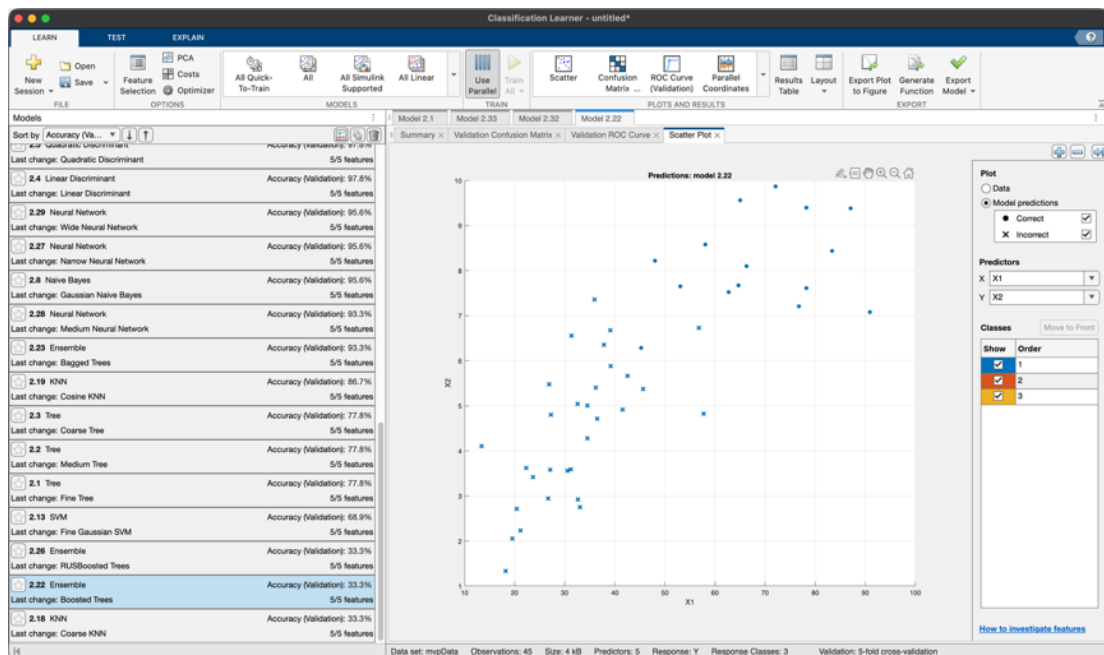


Рисунок 12 – Предсказания модели на плоскости X1–X2

Больше крестиков (ошибок) внутри областей классов; границы между классами размыты, особенно в зоне их перекрытия.

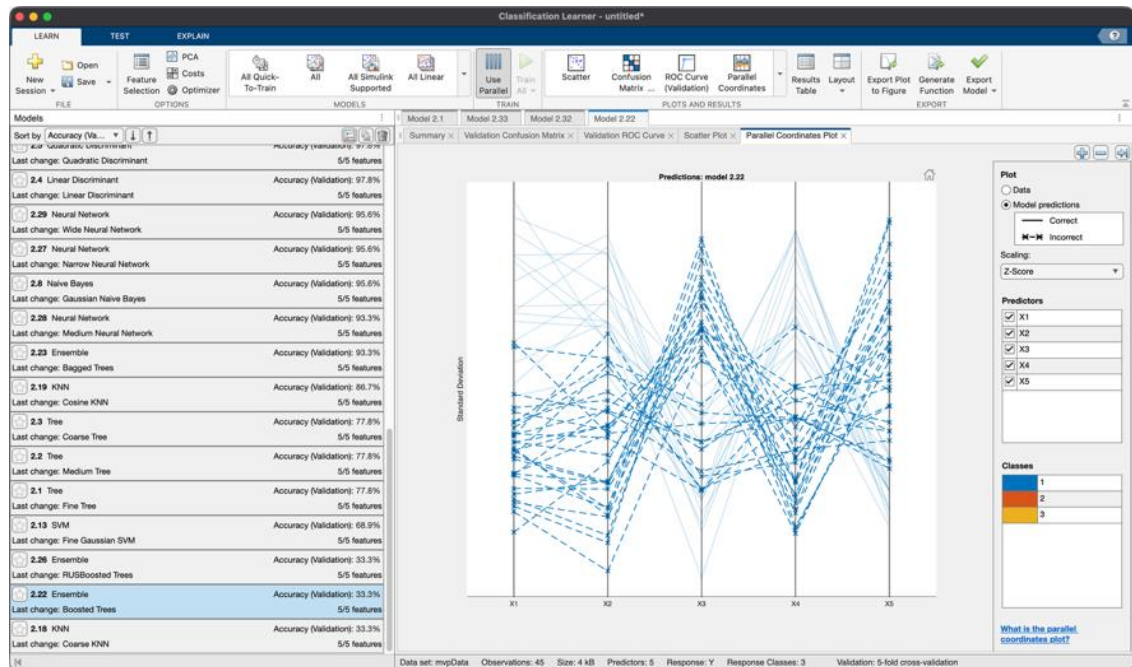


Рисунок 13 – Параллельные координаты

Много пунктирных линий (неверные предсказания). Признаки X3–X5 хуже помогают разделять классы по сравнению с лучшей моделью

Экспорт обученной модели и прогноз на новых данных

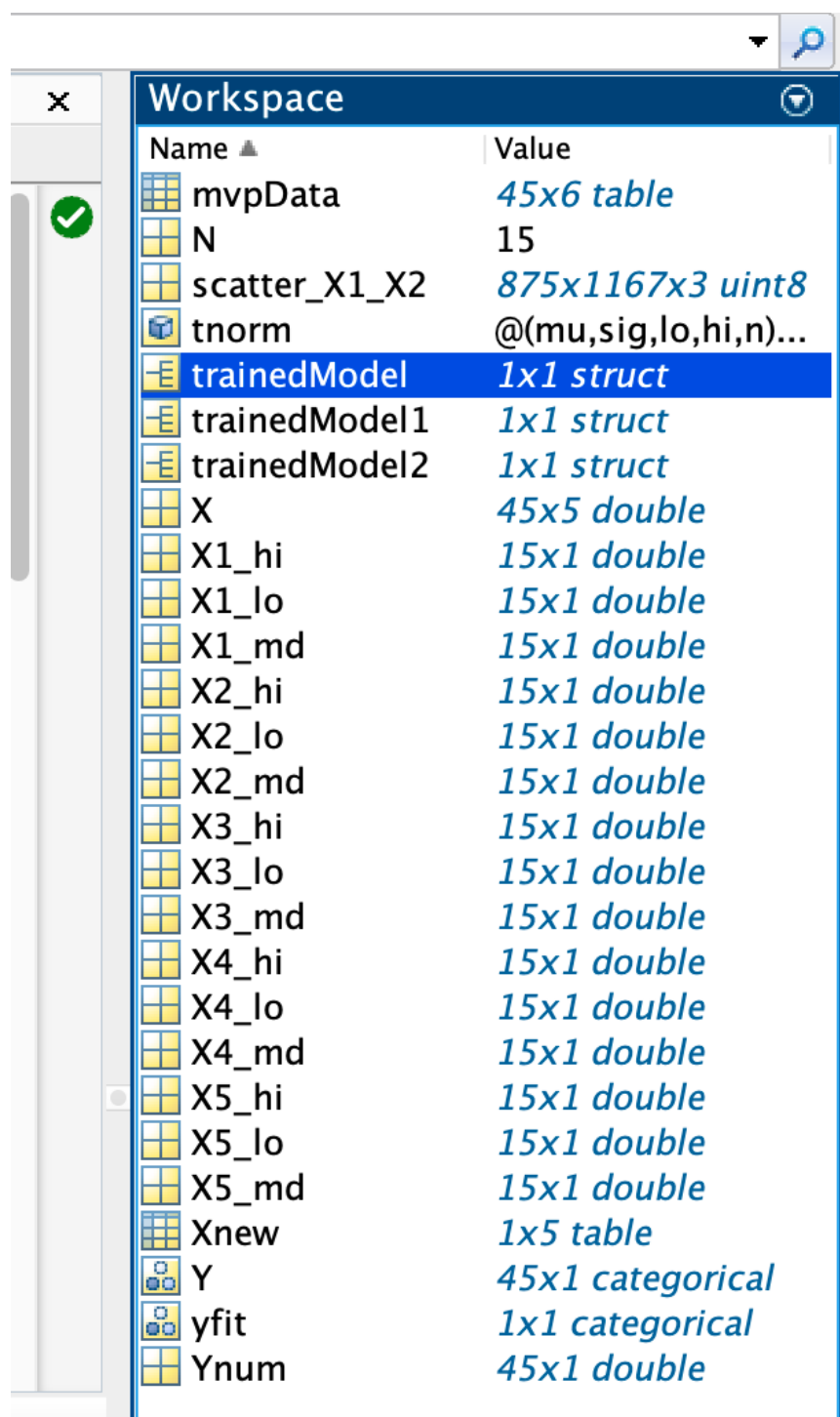
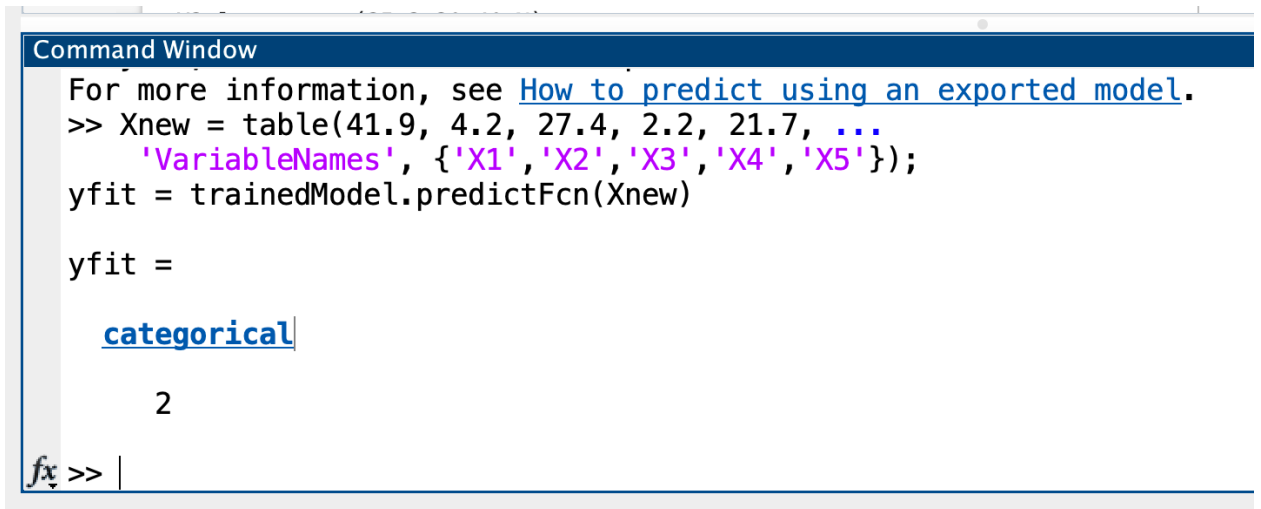


Рисунок 14 – Экспорт лучшей модели в рабочую область MATLAB

На панели Classification Learner выбран лучший классификатор и выполнено Export → Export Compact Model. В Workspace появилась переменная trainedModel (структура 1×1), содержащая всё необходимое для

предсказаний: функцию `predictFcn`, список требуемых предикторов (`RequiredVariables`), имена классов (`ClassNames`), а также сам обученный классификатор (например, `ClassificationSVM`, если модель – SVM). «Compact» означает, что обучающие данные внутрь не включаются → модель лёгкая и готова к применению.



```
Command Window
For more information, see How to predict using an exported model.
>> Xnew = table(41.9, 4.2, 27.4, 2.2, 21.7, ...
    'VariableNames', {'X1','X2','X3','X4','X5'});
yfit = trainedModel.predictFcn(Xnew)

yfit =

    categorical
         2

fx >> |
```

Рисунок 15 – Проверка прогноза на новом наблюдении (Command Window)

Создана таблица `Xnew` с пятью признаками `X1...X5` и теми же именами столбцов, что в обучении, после чего вызвана функция предсказания `yfit = trainedModel.predictFcn(Xnew)`. Результат – `categorical 2`, что соответствует 2-й категории MVP (средняя) согласно принятой шкале: 1 – высокая, 2 – средняя, 3 – низкая.

Подробнее:

Подготовка нового вектора. Новое наблюдение формируется как таблица (обязательно тип `table`) со столбцами `X1...X5` точь-в-точь как в обучающем наборе (названия и типы должны совпадать):

```
Xnew = table(41.9, 4.2, 27.4, 2.2, 21.7, ...
    'VariableNames', {'X1','X2','X3','X4','X5'});
```

Предсказание и интерпретация.

```
yfit = trainedModel.predictFcn(Xnew);
```

Возвращается категориальная метка класса. В нашем случае $y_{fit} = 2$, то есть MVP – средней категории. При необходимости можно посмотреть имена классов из модели:

`trainedModel.ClassNames`

либо `trainedModel.ClassificationSVM.ClassNames`

Экспорт делает модель независимой от приложения-обучателя: её можно вызывать из скриптов/функций, прогонять пакет предсказаний, интегрировать в пайплайн и передавать вместе с проектом. Экспортированная модель хранит ту же нормализацию/преобразования признаков, что были на обучении, поэтому предсказания корректны без дополнительных шагов препроцессинга.

Практическая часть (Задание)

Практическая часть: решение задач классификации (MVP и конкурентное окружение) в MATLAB

1) Создание обучающей выборки: конкурентное окружение (45×6)

Что делаем: создаём синтетическую выборку на основе “5 сил” (5 признаков), по 15 наблюдений на класс (всего 45).

Семантика и шкалы (1...10):

- Audience – влияние потребительской аудитории (выше = лучше).
- NewEntrants – угроза новых конкурентов (выше = хуже).
- Suppliers – зависимость от поставщиков (выше = хуже).
- Substitutes – угроза заменителей (выше = хуже).
- Rivalry – уровень конкурентной борьбы (выше = хуже).

Классы (Y):

1 – благоприятная среда (высокая аудитория, низкие угрозы),

2 – средняя,

3 – неблагоприятная (низкая аудитория, высокие угрозы).

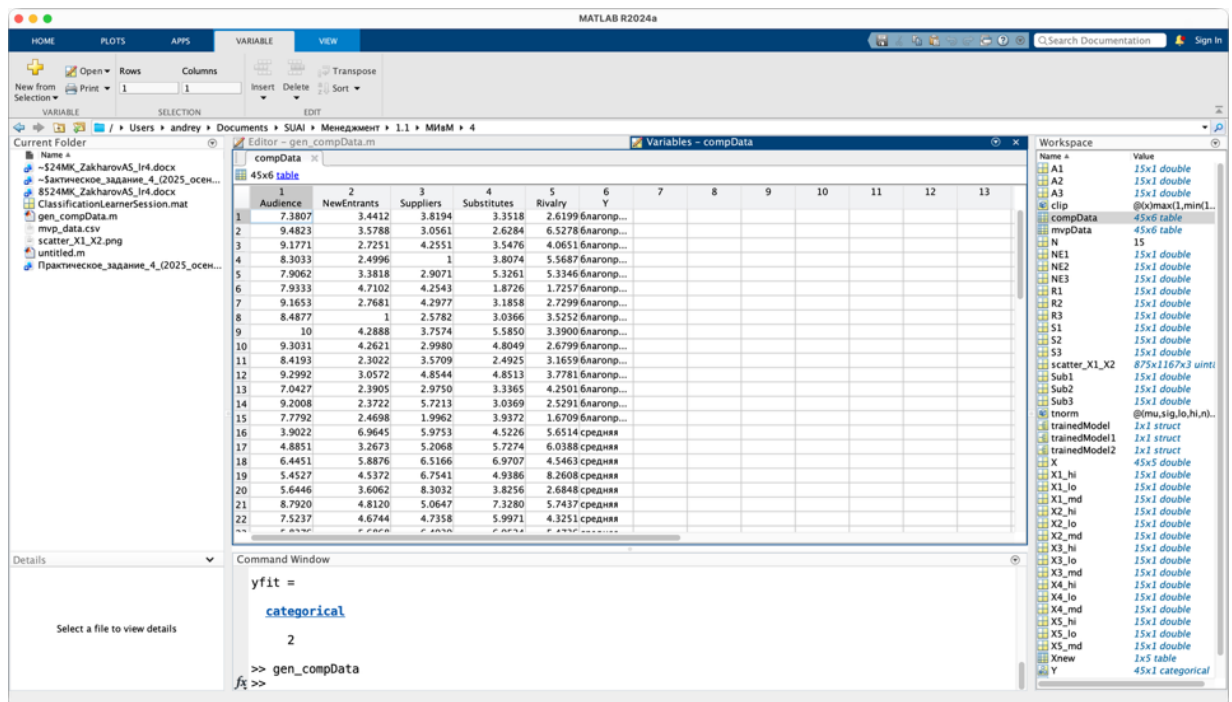


Рисунок 16 – появится переменная compData в Workspace (45×6)

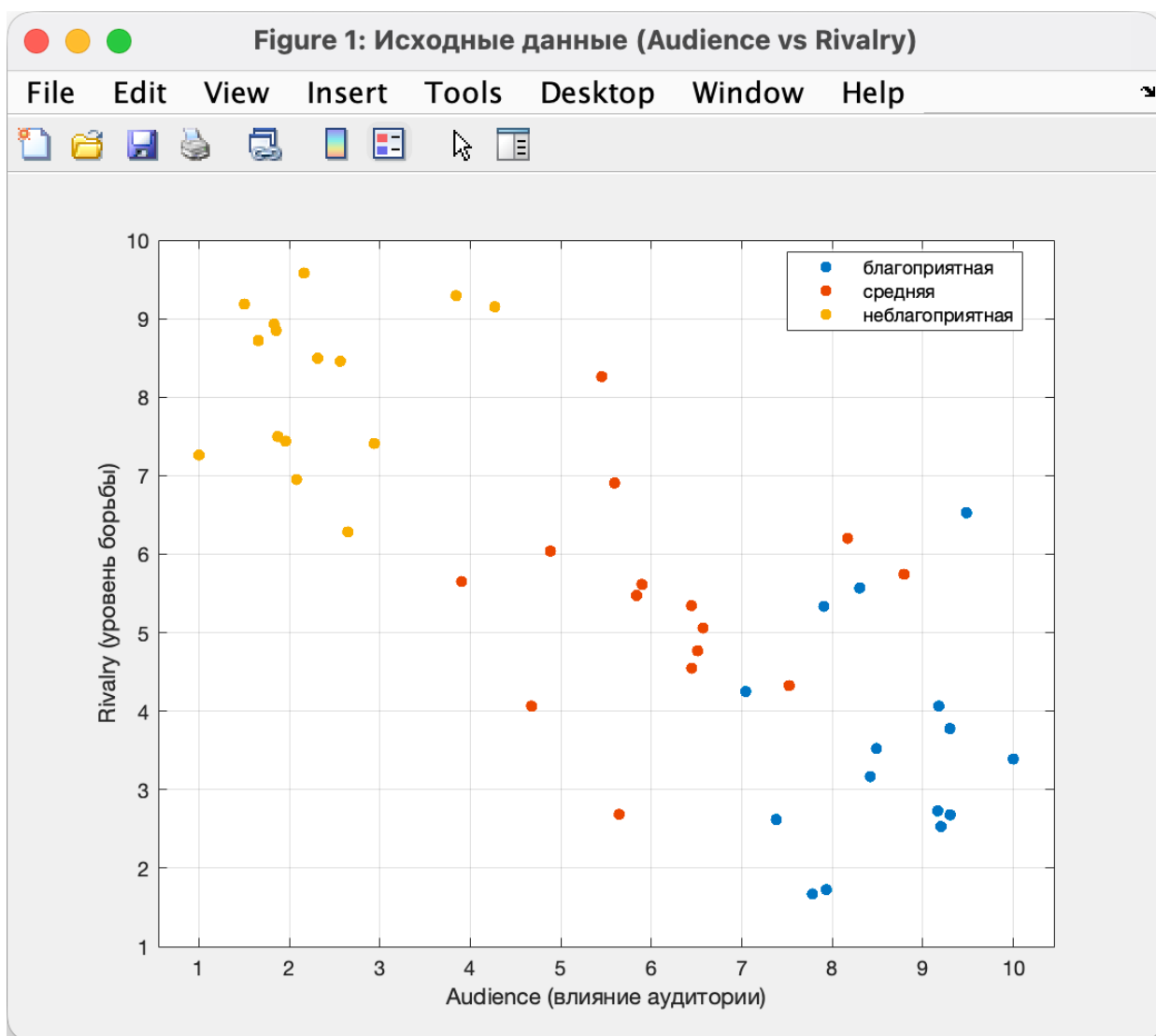


Рисунок 17 – Исходные синтетические данные compData (Audience–Rivalry), цвет – класс Y (1 – благоприятная, 2 – средняя, 3 – неблагоприятная)

Класс 1 характеризуется высокой аудиторией и низкими угрозами; класс 3 – обратный профиль; класс 2 – промежуточные значения с перекрытием.

2) Создание сессии Classification Learner

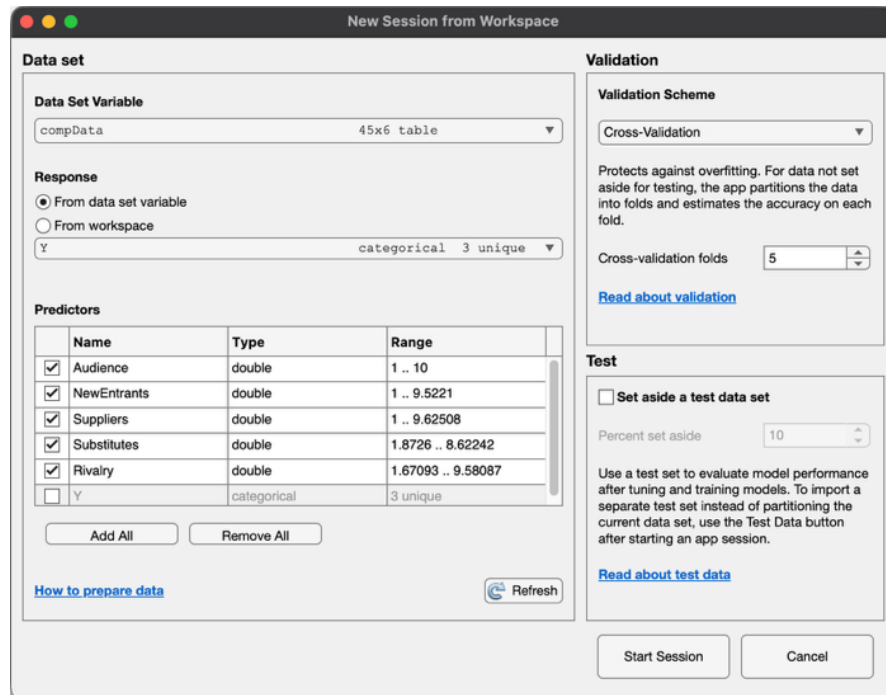


Рисунок 18 – Окно New Session: compData (45×6), Response=Y (3 класса), Predictors=5, схема проверки – 5-fold cross-validation

3) Обучение и выбор лучшей модели

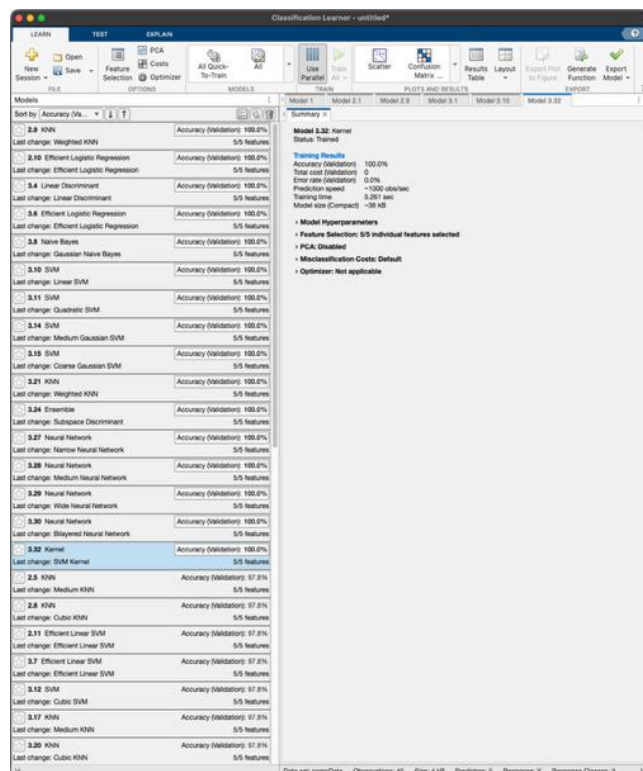


Рисунок 19 – Список обученных моделей, отсортированных по Аккураси

Несколько алгоритмов (KNN, логистическая регрессия, ЛДА, SVM) показали 100% точность по 5-кратной кросс-валидации на синтетическом наборе; это объясняется хорошей разделяемостью классов.

В качестве итоговой выбрана SVM (Kernel) как более устойчивая к нелинейным границам; результаты подтверждены матрицей ошибок и ROC-кривыми ($AUC \approx 1$).

4) Анализ качества

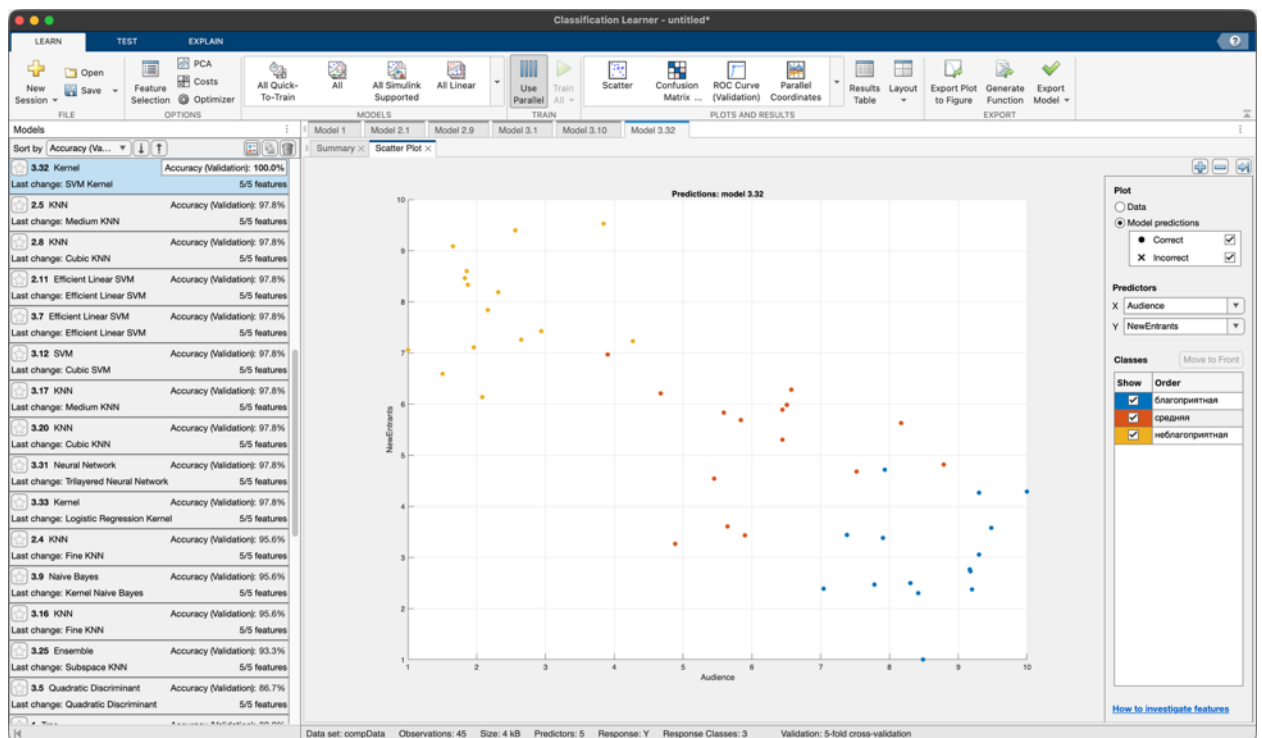


Рисунок 20 – Scatter (Model predictions): «цвет – предсказанный класс, крестики – ошибки, визуально подтверждает/показывает границы классов

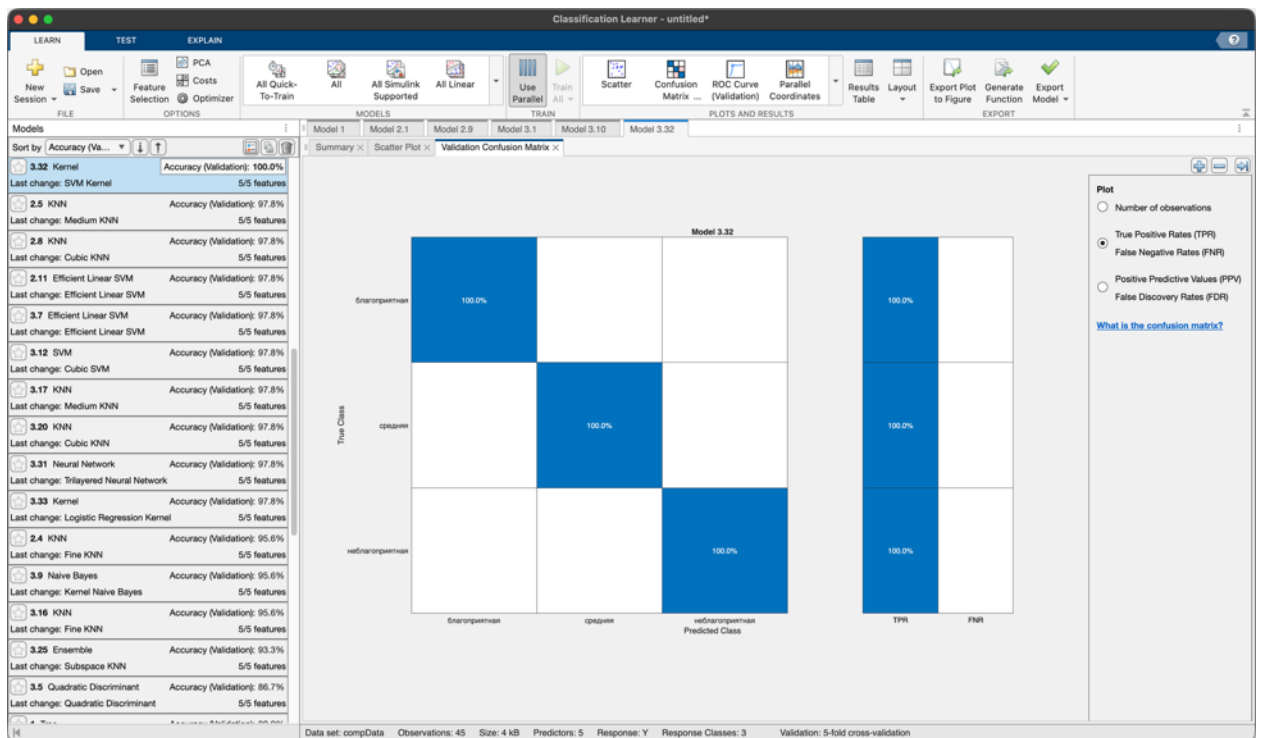


Рисунок 21 – Матрица ошибок (5-fold CV): «все/большинство точек на диагонали; справа приведены TPR/FNR по классам

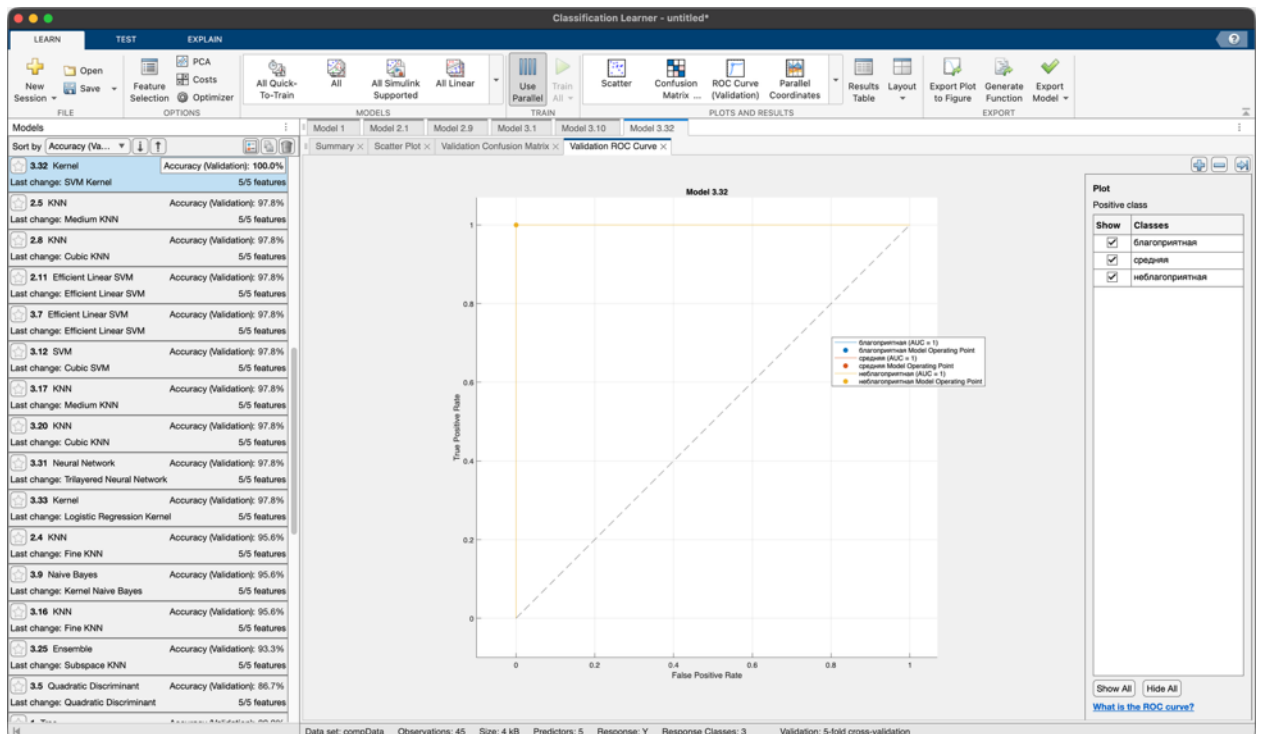


Рисунок 22 – кривые выше диагонали случайного угадывания; AUC = 1 для лучшей модели

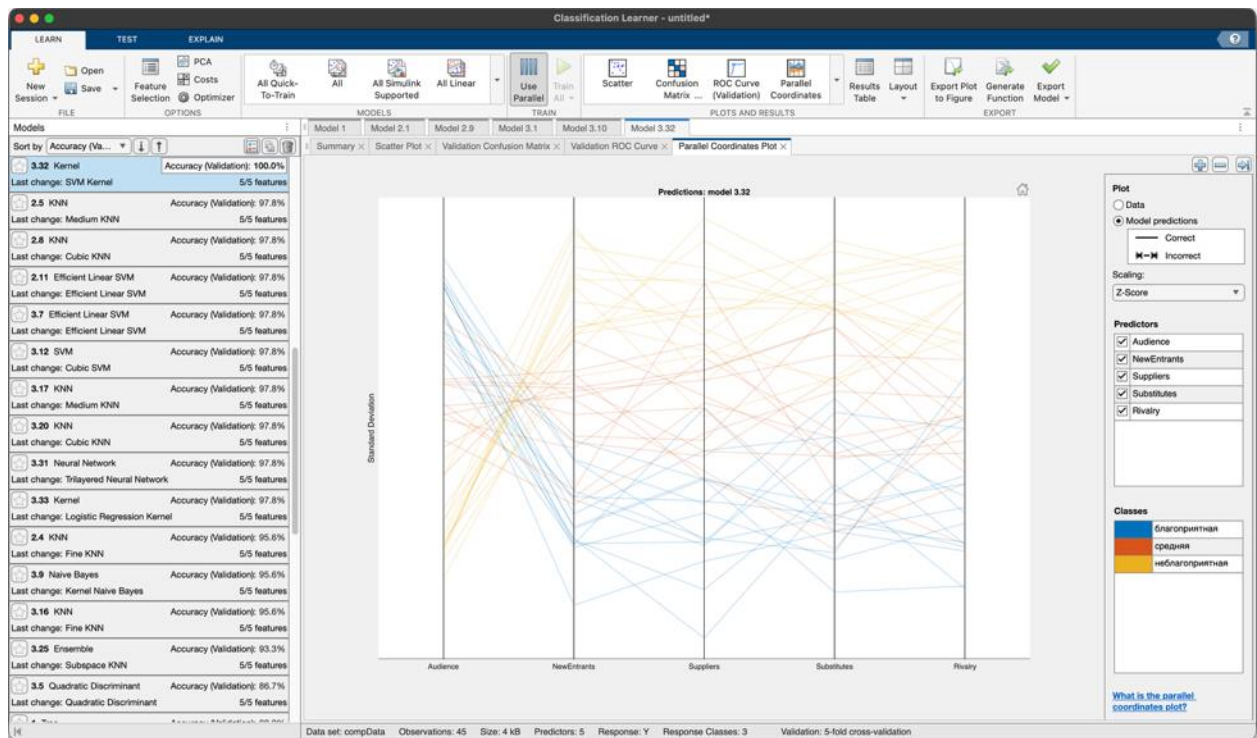
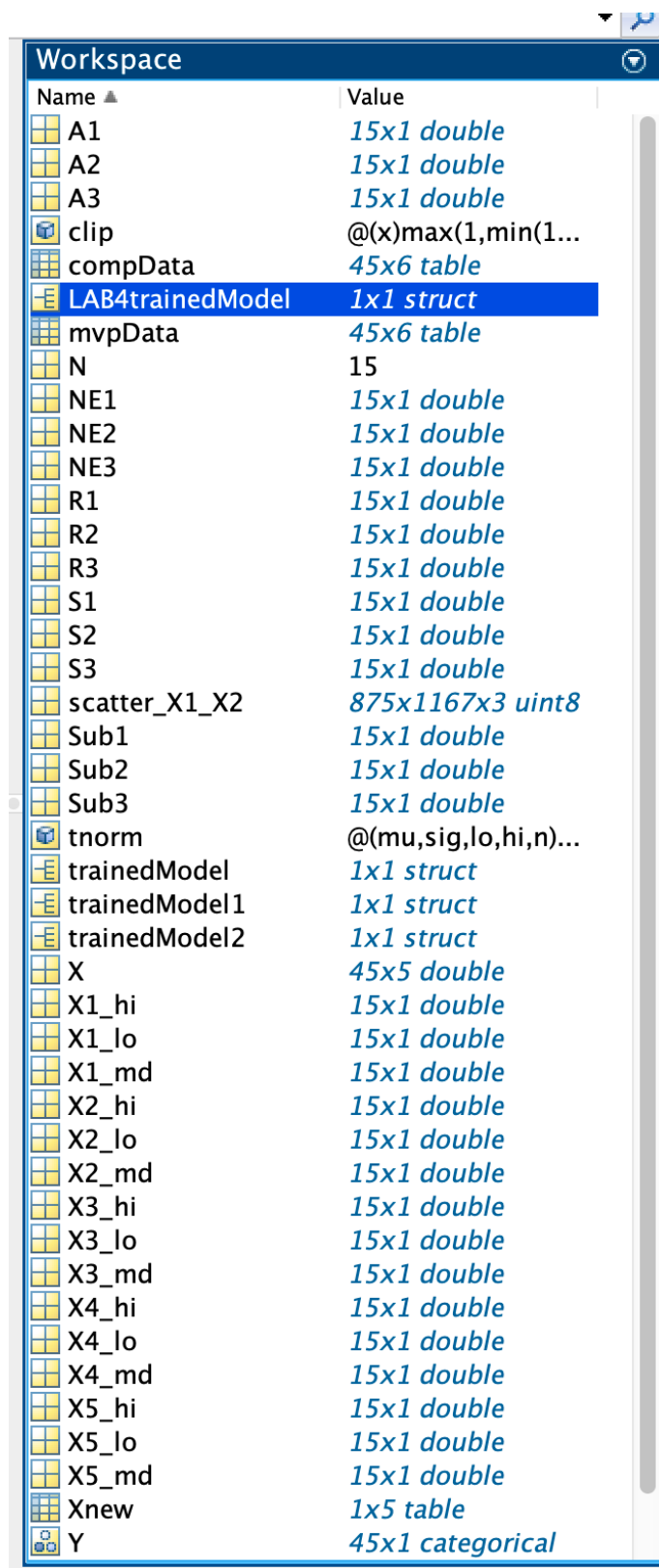


Рисунок 23 – класс 1 – высокий Audience и низкие NE/Sup/Sub/Rivalry; класс 3 – наоборот; класс 2 – промежуточный профиль; пунктир – ошибки, отсутствуют

5) Экспорт лучшей модели и проверка прогноза

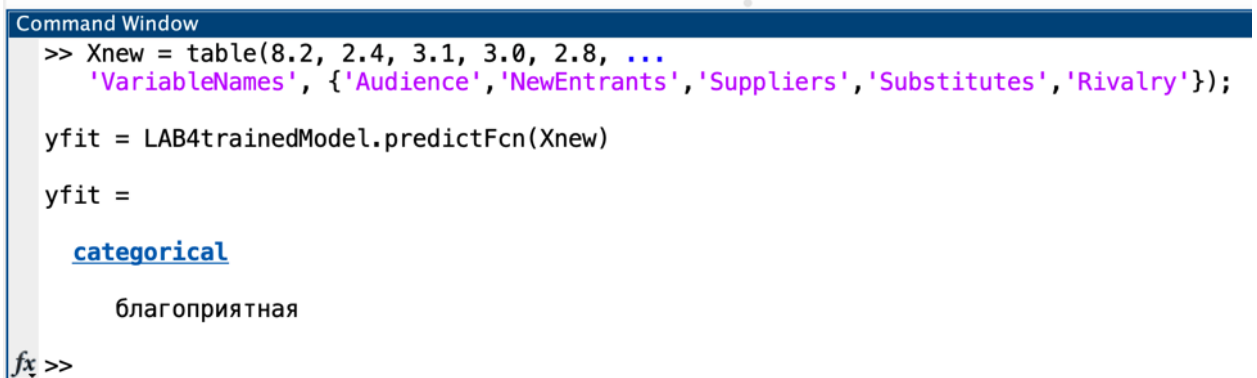


The screenshot shows the MATLAB Workspace window with a list of variables and their data types. The variable 'LAB4trainedModel' is highlighted in blue. The list includes various data matrices, tables, and categorical arrays.

Name	Value
A1	15x1 double
A2	15x1 double
A3	15x1 double
clip	@(x)max(1,min(1...
compData	45x6 table
LAB4trainedModel	1x1 struct
mvpData	45x6 table
N	15
NE1	15x1 double
NE2	15x1 double
NE3	15x1 double
R1	15x1 double
R2	15x1 double
R3	15x1 double
S1	15x1 double
S2	15x1 double
S3	15x1 double
scatter_X1_X2	875x1167x3 uint8
Sub1	15x1 double
Sub2	15x1 double
Sub3	15x1 double
tnorm	@(mu,sig,lo,hi,n)...
trainedModel	1x1 struct
trainedModel1	1x1 struct
trainedModel2	1x1 struct
X	45x5 double
X1_hi	15x1 double
X1_lo	15x1 double
X1_md	15x1 double
X2_hi	15x1 double
X2_lo	15x1 double
X2_md	15x1 double
X3_hi	15x1 double
X3_lo	15x1 double
X3_md	15x1 double
X4_hi	15x1 double
X4_lo	15x1 double
X4_md	15x1 double
X5_hi	15x1 double
X5_lo	15x1 double
X5_md	15x1 double
Xnew	1x5 table
Y	45x1 categorical

Рисунок 24 – Экспорт compact-модели в Workspace (переменная LAB4trainedModel)

```
Xnew = table(8.2, 2.4, 3.1, 3.0, 2.8, ...  
            'VariableNames',  
            {'Audience','NewEntrants','Suppliers','Substitutes','Rivalry'});  
yfit = LAB4trainedModel.predictFcn(Xnew)
```



```
Command Window  
>> Xnew = table(8.2, 2.4, 3.1, 3.0, 2.8, ...  
    'VariableNames', {'Audience','NewEntrants','Suppliers','Substitutes','Rivalry'});  
yfit = LAB4trainedModel.predictFcn(Xnew)  
yfit =  
    categorical  
    благоприятная  
fx >>
```

Рисунок 25 – Command Window: создание Xnew и вызов LAB4trainedModel.predictFcn(Xnew); получена метка класса (categorical)

Вывод:

На обеих синтетических выборках (MVP и конкурентное окружение, 45×6) несколько алгоритмов (SVM с ядром, ЛДА, логистическая регрессия, местами KNN) достигли 100% точности по 5-кратной кросс-валидации, что объясняется хорошей разделимостью данных (логически противоположные профили классов). Матрицы ошибок диагональные, ROC-кривые прижаты к левому верхнему углу (AUC≈1). В качестве итоговой модели выбрана SVM (Kernel) как устойчивая к нелинейным границам; экспорт compact-модели позволил корректно классифицировать новые наблюдения через predictFcn.

Листинг

Для первой, теоретической части

```
%% ПЗ-4. Генерация обучающей выборки (45 наблюдений = по 15 на класс)
rng(2025);

% Удобная функция "усечённое нормальное"
tnorm = @(mu,sig,lo,hi,n) random(truncate(makedist('Normal',mu,sig),lo,hi),n,1);

N = 15; % по классу

% Класс 1 (высокая): много скачиваний, рейтинг высокий, размер умеренный,
%      покупатели высокий %, отток низкий
X1_hi = tnorm(65,15,45,100,N); % тыс. скачиваний
X2_hi = tnorm(8.2,1.0,6.0,10.0,N); % рейтинг 1..10
X3_hi = tnorm(22,5,15,30,N); % размер, МБ
X4_hi = tnorm(7.5,1.0,5.0,10.0,N); % % покупателей 1..10
X5_hi = tnorm(8.5,2.0,5.0,12.0,N); % отток, % 5..30 (низкий)

% Класс 2 (средняя): всё среднее
X1_md = tnorm(45,12,30,70,N);
X2_md = tnorm(6.0,1.2,4.0,8.5,N);
X3_md = tnorm(27,5,20,35,N);
X4_md = tnorm(5.0,1.2,3.0,7.5,N);
X5_md = tnorm(15,3.0,10,20,N);

% Класс 3 (низкая): мало скачиваний, рейтинг низкий, размер большой,
%      покупатели низкий %, отток высокий
X1_lo = tnorm(25,8,10,40,N);
X2_lo = tnorm(3.0,1.0,1.0,5.5,N);
X3_lo = tnorm(35,3,30,40,N);
X4_lo = tnorm(2.5,1.0,1.0,4.5,N);
X5_lo = tnorm(22,4.0,16,30,N);

% Собираем матрицу
X = [ ...
    X1_hi X2_hi X3_hi X4_hi X5_hi; % класс 1
    X1_md X2_md X3_md X4_md X5_md; % класс 2
    X1_lo X2_lo X3_lo X4_lo X5_lo]; % класс 3

Ynum = [ones(N,1); 2*ones(N,1); 3*ones(N,1)]; % 1/2/3
Y = categorical(Ynum,[1 2 3],{'1','2','3'}); % категориально

% В таблицу с нужными именами столбцов
mvpData = array2table(X, 'VariableNames',{'X1','X2','X3','X4','X5'});
mvpData.Y = Y; % отклик — категориальный столбец

% Быстрый скаттер X1–X2 (как в методичке)
figure('Color','w'); hold on;
gscatter(mvpData.X1, mvpData.X2, mvpData.Y, 'rgb', 'o^s', 8, 'on');
xlabel('X1: скачивания (тыс.)'); ylabel('X2: рейтинг (1..10)');
title('Исходные данные (X1–X2)');
grid on; box on;
saveas(gcf,'scatter_X1_X2.png'); close;

% (опционально) сохранить CSV для архива
writetable(mvpData,'mvp_data.csv');
disp('Готово: в Workspace есть таблица mvpData (X1..X5,Y).');
```

Для второй части, задания

```
%% Генерация данных для задачи "Конкурентное окружение"
rng(2025); % можно убрать для полностью случайного набора
N = 15; % по 15 наблюдений на класс
clip = @(x) max(1, min(10, x)); % ограничение в диапазоне [1..10]

% Класс 1 (благоприятная): высокая аудитория, низкие угрозы
A1 = clip( 8.5 + 1.0*randn(N,1)); % Audience (высокая)
NE1= clip( 3.0 + 1.0*randn(N,1)); % NewEntrants (низкая)
S1 = clip( 3.0 + 1.0*randn(N,1)); % Suppliers (низкая зависимость)
Sub1= clip( 3.5 + 1.0*randn(N,1)); % Substitutes (низкая)
R1 = clip( 3.0 + 1.2*randn(N,1)); % Rivalry (низкая)

% Класс 3 (неблагоприятная): низкая аудитория, высокие угрозы
A3 = clip( 3.0 + 1.0*randn(N,1));
NE3= clip( 8.0 + 1.0*randn(N,1));
S3 = clip( 8.0 + 1.0*randn(N,1));
Sub3= clip( 7.8 + 1.0*randn(N,1));
R3 = clip( 8.5 + 1.0*randn(N,1));

% Класс 2 (средняя): промежуточные значения, создаём перекрытие классов
A2 = clip( 6.0 + 1.2*randn(N,1));
NE2= clip( 5.0 + 1.2*randn(N,1));
S2 = clip( 5.5 + 1.2*randn(N,1));
Sub2= clip( 5.0 + 1.3*randn(N,1));
R2 = clip( 5.0 + 1.3*randn(N,1));

% Собираем в одну матрицу (порядок классов: 1, 2, 3)
X = [ [A1; A2; A3], ...
      [NE1; NE2; NE3], ...
      [S1; S2; S3], ...
      [Sub1; Sub2; Sub3], ...
      [R1; R2; R3] ];

% Метки классов (categorical с подписями)
Y = categorical([ones(N,1); 2*ones(N,1); 3*ones(N,1)], ...
               [1 2 3], {'благоприятная','средняя','неблагоприятная'});

% Таблица предикторов + целевая переменная
compData = array2table(X, 'VariableNames', ...
                      {'Audience','NewEntrants','Suppliers','Substitutes','Rivalry'});
compData.Y = Y;

% Быстрый предпросмотр в 2D (Аудитория vs Конкуренция)
figure('Name','Исходные данные (Audience vs Rivalry)');
gscatter(compData.Audience, compData.Rivalry, compData.Y);
xlabel('Audience (влияние аудитории)'); ylabel('Rivalry (уровень борьбы)');
grid on;

% (необязательно) Сохранить таблицу на диск
% writetable(compData, 'compData.csv');
```